INF 345 — Fundamentals of DevOps

Lecture 9: Idempotence & Testing with Molecule

Adil Akhmetov · Lesson 9



Recap — Lesson 8 (Playbooks, Roles & Variables)

- A role default says `app_port: 8080`, `group_vars/web.yml` says `8081`. Which wins? (`8081` — role defaults are the weakest variables)
- You put `app_port` in `roles/webapp/vars/main.yml` and the `group_vars` override stopped working. Why? (role `vars/` beat inventory variables — values meant to be overridden go in `defaults/`)
- Inside a role, how do you point a `template` task at `roles/webapp/templates/app.conf.j2`? (`src: app.conf.j2` — the role knows where to look)
- The handler runs on every single run. What's wrong? (a task that notifies it always reports `changed` — it isn't idempotent. Today's topic.)

Today's agenda

- ☐ Idempotence: what it means, and why automation lives or dies by it
- ☐ `ok` vs `changed` — how Ansible decides, and how to read the recap
- ☐ Why `shell` / `command` break it — and four ways to fix them
- ☐ Check mode: `--check` and `--diff` — a dry run before the real run
- ☐ `ansible-lint` in one slide
- ☐ Molecule: test a role the same way every time
- ☐ Worked example: a Molecule scenario for last week's `webapp` role
- ☐ Straight into today's practice

The scenario

Your `webapp` role ran fine on Monday. On Wednesday someone runs it again "just to be sure" — and the site reloads, a config line appears twice, and a log file is wiped. On Friday a teammate edits a task, nobody runs it anywhere before production, and it breaks three servers.

Today: automation you can run a hundred times safely — and a test that proves it before it reaches a real server.

Block 1

Idempotence

Idempotent: running it again changes nothing

- **Idempotent** = applying it once or ten times gives the **same end state**. The second run finds everything already right and does nothing.
- Ansible describes the **desired state** ("this file exists, with this content, mode 0644"), not the steps to get there.
- A module first **checks** the current state, then acts **only if it differs** — that's where `ok` and `changed` come from.
- Why it matters:
  - you can re-run a playbook to *repair* drift, without fear
  - a failed run can simply be run again
  - "changed" means something: a real difference, worth a handler

```
$ ansible-playbook site.yml      # first run
localhost : ok=6  changed=5  unreachable=0  failed=0
$ ansible-playbook site.yml      # second run
localhost : ok=6  changed=0  unreachable=0  failed=0
```

The **second run** is the test: `changed=0`.

`ok` vs `changed`

Status	Meaning	Example
`ok`	State was already right — nothing done	the directory exists with mode 0755
`changed`	Ansible changed something on the host	the file was created, or its content differed
`skipping`	`when:` was false	a RedHat-only task on Ubuntu
`failed`	The task failed; this host stops	a template with an undefined variable

- `changed` is what **triggers handlers** — so a task that lies about "changed" makes every handler run every time.
- The play **recap** at the end sums it up per host: `ok=6 changed=0 failed=0` is the result you want from a re-run.

Modules that are idempotent out of the box

```yaml
- name: App directory
  ansible.builtin.file:
    path: "{{ app_root }}"
    state: directory
    mode: "0755"

- name: APP_ENV in .env
  ansible.builtin.lineinfile:
    path: "{{ app_root }}/.env"
    line: "APP_ENV={{ app_env }}"
    regexp: "^APP_ENV="
    create: true
```

- `file`, `copy`, `template`, `lineinfile`, `package`, `service`, `user` … each one compares **before** acting.
- `copy` / `template` compare a checksum: same content → `ok`.
- `lineinfile` with `regexp:` replaces the line instead of adding a second copy.
- `package: state=present` installs once; `state=latest` upgrades — which is *not* idempotent over time. Prefer `present`.

Block 2

`shell` and `command`: the idempotence killers

Why `shell` breaks idempotence

```
- name: App directory
  ansible.builtin.shell: mkdir -p /tmp/app

- name: APP_ENV in .env
  ansible.builtin.shell: echo "APP_ENV=prod" >> /tmp/app/.

- name: Log file
  ansible.builtin.command: touch /tmp/app/app.log
```

- Ansible can't know what a shell command did, so it **always** reports `changed`.
- `mkdir -p` is harmless — but still `changed` every run.
- `echo >>` **appends again** every run: the file grows forever.
- `touch` updates the timestamp every run — `changed` forever.
- Result: `changed=3` on every run, handlers firing for no reason, and a broken `.env`.

Fix 1: use the real module

Before

```
- ansible.builtin.shell: mkdir -p /tmp/app
- ansible.builtin.shell: echo "APP_ENV=prod" >> /tmp/app/.
- ansible.builtin.command: touch /tmp/app/app.log
```

After

```
- ansible.builtin.file:
    path: /tmp/app
    state: directory
- ansible.builtin.lineinfile:
    path: /tmp/app/.env
    line: APP_ENV=prod
    create: true
- ansible.builtin.file:
    path: /tmp/app/app.log
    state: touch
    modification_time: preserve
    access_time: preserve
```

Always the first choice. There's a module for almost everything — search `ansible-doc -l | grep <word>` before reaching for `shell`.

Fix 2: `creates:` / `removes:`

When a command produces a file, tell Ansible which one:

```
- name: Initialise the database (once)
  ansible.builtin.command: /opt/app/bin/init-db --data /var/lib/app
  args:
    creates: /var/lib/app/initialized      # skip if this file exists

- name: Remove the old cache (only if it's still there)
  ansible.builtin.command: rm -rf /var/cache/app-old
  args:
    removes: /var/cache/app-old             # skip if it's already gone
```

- `creates:` — the command runs **only if the path doesn't exist**; otherwise the task is `ok` and nothing runs.
- `removes:` — the opposite: runs **only if the path exists**.
- Works for `command` and `shell`.

Fix 3: `changed_when` / `failed_when`

```yaml
- name: Read the app version
  ansible.builtin.command: /opt/app/bin/app --version
  register: app_version
  changed_when: false                # a read-only command

- name: Apply migrations
  ansible.builtin.command: /opt/app/bin/migrate
  register: migrate
  changed_when: "'Applied' in migrate.stdout"
  failed_when: migrate.rc not in [0, 2]
```

- `changed_when: false` — this command only **reads**; never report a change.
- `changed_when: <expr>` — decide from the output (`stdout` , `rc`) whether it really changed something.
- `failed_when:` — decide what counts as failure (here: exit code 2 means "nothing to do", not an error).
- No `{{ }}` — like `when:` , these take a bare expression.

Fix 4: check first, act only if needed

```yaml
- name: Is the app user in the docker group?
  ansible.builtin.command: id -nG deploy
  register: groups_out
  changed_when: false

- name: Add the app user to the docker group
  ansible.builtin.user:
    name: deploy
    groups: docker
    append: true
  when: "'docker' not in groups_out.stdout.split()"
```

- A read-only check (`changed_when: false`) + `register` + `when:` .
- Here the `user` module alone would already be idempotent — the pattern is for commands that have **no** module.
- Order of preference: **module** → `creates` / `removes` → `changed_when` → **check** + `when` .

Quick check

`ansible.builtin.shell: echo "APP_ENV=prod" >> .env` runs on every play. What does `.env` look like after three runs, and what does the recap say on run three?

Three `APP_ENV=prod` **lines**, and `changed=1` every time. Replace it with `lineinfile` (`line:` + `regexp: "^APP_ENV="`): one line, and `ok` after the first run.

Block 3

Check mode: a dry run

`--check` and `--diff`

```
ansible-playbook site.yml --check          # what WOULD ch
ansible-playbook site.yml --check --diff   # …and show the
ansible-playbook site.yml --diff           # real run, sho
```

```
TASK [webapp : Site config] ***************
--- before: /tmp/inf345-p09/app/app.conf
+++ after: /tmp/inf345-p09/app/app.conf
@@ -1,3 +1,3 @@
 name = inf345
-port = 8080
+port = 8081
changed: [localhost]
```

- `--check` : modules **report** what they would do, but change nothing.
- `--diff` : show the before/after of files that change (`template`, `copy`, `lineinfile`…).
- Together: review a change before it touches a server.
- Idempotent playbook + a converged host → `--check` reports `changed=0`.

Check mode and `shell`

```yaml
- name: Read the app version
  ansible.builtin.command: /opt/app/bin/app --version
  register: app_version
  changed_when: false
  check_mode: false        # safe to run for real, even in

- name: Show it
  ansible.builtin.debug:
    msg: "{{ app_version.stdout }}"
```

- In `--check`, `command` / `shell` are **skipped** — Ansible can't predict them. Their `register` is empty, so later tasks may break.
- `check_mode: false` on a **read-only** command: it runs even in check mode.
- `check_mode: true` on a task: it **always** runs in check mode (never changes anything).
- One more reason to prefer real modules: they support check mode for free.

Handlers and idempotence

- A handler runs only if a task that **notifies** it reported `changed`.
- Idempotent tasks → on a re-run nothing changes → **no handler**, no needless restarts.
- A non-idempotent task that notifies a handler restarts your service on **every run** — users notice that.
- So "the handler runs every time" is almost always a non-idempotent task, not a handler problem.

```
RUNNING HANDLER [webapp : Reload app] ***   ← on run 2? look for the task that said "changed"
```

`ansible-lint`: catch it before you run it

```
pipx install ansible-lint
ansible-lint site.yml roles/webapp
```

```
no-changed-when: Commands should not change things
  if nothing needs doing.
roles/webapp/tasks/main.yml:12 Task/Handler: Read version

command-instead-of-module: mkdir used in place of
  file module
roles/webapp/tasks/main.yml:3 Task/Handler: App directory
```

- A linter for playbooks and roles: style **and** the common bugs.
- `no-changed-when` — a `command` / `shell` with no `changed_when`.
- `command-instead-of-module` — `mkdir`, `touch`, `chmod`… in a command.
- `risky-file-permissions` — a file created without `mode:`.
- Run it in CI, next to the tests.

Block 4

Molecule: testing a role

Why test a role at all?

- "It worked on my laptop once" is not a test.
- Every change to a role can break: a typo in a variable, a task that's no longer idempotent, a template that renders wrong.
- A test should answer three questions, **every time**, automatically:
  1. Does the role **run** without errors?
  2. Is it **idempotent** — does a second run change nothing?
  3. Is the result **right** — files, contents, modes?

Molecule answers all three. It's the standard tool for testing Ansible roles (part of the Ansible project):

```
pipx install molecule
# or alongside ansible-core:
pip install ansible-core molecule
```

A scenario: three files

```
roles/webapp/            (or the repo root)
├── tasks/ defaults/ handlers/ …
└── molecule/
    └── default/              ← the scenario's name
        ├── molecule.yml      ← where & how to test
        ├── converge.yml      ← playbook that applies the
        └── verify.yml        ← checks the result
```

- A **scenario** = one way of testing. `default` is the one Molecule runs unless told otherwise.
- `molecule.yml` — the **driver** (where it runs) and the platforms (which machines).
- `converge.yml` — a small playbook that applies your role.
- `verify.yml` — a playbook of **assertions** about the result.

The test sequence

```
molecule test
```

Step	What happens
`dependency`	install required roles/collections
`syntax`	`--syntax-check` on converge.yml
`create`	create the test machines
`prepare`	optional setup before the role
`converge`	**run the role**
`idempotence`	**run it again — fail if anything changed**
`verify`	**run verify.yml**
`destroy`	throw the machines away

- `idempotence` is built in: the second converge must report `changed=0`, or the test **fails** and lists the guilty tasks.
- Step by step while you work:
  - `molecule converge` — apply the role (keep the machine)
  - `molecule idempotence` — run it again, check nothing changed
  - `molecule verify` — run the checks
  - `molecule test` — the whole sequence, from scratch

Drivers: where the test runs

Container driver (docker / podman)

```
# molecule/default/molecule.yml
driver:
  name: docker
platforms:
  - name: instance
    image: docker.io/geerlingguy/docker-ubuntu2404-ansible
    pre_build_image: true
```

A fresh container per test: clean, close to a real server. Needs Docker or Podman and `molecule-plugins[docker]`.

Default driver, unmanaged (today's practice)

```
# molecule/default/molecule.yml
driver:
  name: default
  options:
    managed: false
platforms:
  - name: instance
provisioner:
  name: ansible
  inventory:
    host_vars:
      instance:
        ansible_connection: local
```

No containers: Molecule runs against **this machine** — like your practices so far. Fast, works on a CI runner as-is.

`converge.yml` : apply the role

```yaml
# molecule/default/converge.yml
- name: Converge
  hosts: all
  gather_facts: false
  roles:
    - role: webapp
```

- `hosts: all` — every platform in `molecule.yml` (here: `instance`).
- The role is found through `roles_path` — when `molecule/` sits at the repo root next to `roles/`, point to it in `molecule.yml`:

```yaml
provisioner:
  name: ansible
  env:
    ANSIBLE_ROLES_PATH: ../../roles
```

- Pass test-specific variables here (`vars:`) if you need them.

`verify.yml` : assertions about the result

```yaml
# molecule/default/verify.yml
- name: Verify
  hosts: all
  gather_facts: false
  tasks:
    - name: Stat the app directory
      ansible.builtin.stat:
        path: /tmp/inf345-p09/app
      register: app_dir

    - name: The directory exists with mode 0755
      ansible.builtin.assert:
        that:
          - app_dir.stat.isdir
          - app_dir.stat.mode == '0755'

    - name: Read .env
      ansible.builtin.slurp:
        src: /tmp/inf345-p09/app/.env
      register: env_file
    - ansible.builtin.assert:
        that: "(env_file.content | b64decode).splitlines()
```

- `verify.yml` is just a playbook — Molecule's default verifier is **Ansible** itself.
- `stat` → does it exist? type? mode? owner?
- `slurp` → read a file's content (base64 — decode with `b64decode`).
- `assert` with `that:` → a list of expressions that must all be true; `fail_msg:` for a readable error.
- Verify **outcomes**, not tasks: "the file has one APP_ENV line", not "the lineinfile task ran".

Quick check

`molecule converge` succeeds. `molecule test` fails at the `idempotence` step and names the task "Log file". What do you look at?

That task reports `changed` **on the second run** — probably a `command: touch` or a `file: state=touch` without `preserve`. Make it idempotent; don't touch the test.

Worked example

Last week’s `webapp` role — and a teammate’s "small addition" that quietly broke it. Find the non-idempotent tasks, fix them, and add a Molecule scenario so it can’t happen again.

The teammate's addition

```
- name: Release marker
  ansible.builtin.shell: date > {{ app_root }}/RELEASE

- name: APP_ENV in .env
  ansible.builtin.shell: echo "APP_ENV={{ app_env }}" >> {{ app_root }}/.env

- name: Show the config
  ansible.builtin.command: cat {{ app_root }}/app.conf
  register: conf
```

```
$ ansible-playbook site.yml      # second run
localhost : ok=8  changed=3  unreachable=0  failed=0
RUNNING HANDLER [webapp : Reload app]     ← and it reloads, every time
```

The fix

```yaml
- name: Release marker (written once)
  ansible.builtin.copy:
    content: "release {{ app_version }}\n"
    dest: "{{ app_root }}/RELEASE"
    mode: "0644"

- name: APP_ENV in .env
  ansible.builtin.lineinfile:
    path: "{{ app_root }}/.env"
    regexp: "^APP_ENV="
    line: "APP_ENV={{ app_env }}"
    create: true
    mode: "0644"

- name: Show the config
  ansible.builtin.command: cat {{ app_root }}/app.conf
  register: conf
  changed_when: false
  check_mode: false
```

- `date >` changed the file every run → write content that only changes when the **release** changes (a variable).
- `echo >>` appended forever → `lineinfile` with `regexp:`.
- `cat` only reads → `changed_when: false` (and `check_mode: false` so it also runs in `--check`).
- Second run: `changed=0`, and the handler stays quiet.

The scenario for it

`molecule/default/molecule.yml`

```yaml
driver:
  name: default
  options:
    managed: false
platforms:
  - name: instance
provisioner:
  name: ansible
  env:
    ANSIBLE_ROLES_PATH: ../../roles
  inventory:
    host_vars:
      instance:
        ansible_connection: local
```

`molecule/default/verify.yml` (excerpt)

```yaml
- name: Verify
  hosts: all
  gather_facts: false
  tasks:
    - ansible.builtin.slurp:
        src: /tmp/inf345-p09/app/.env
      register: env_file
    - ansible.builtin.assert:
        that:
          - "(env_file.content | b64decode).splitlines()
             | select('match', '^APP_ENV=') | list | lengt
        fail_msg: ".env must have exactly one APP_ENV line
```

Run it

```
molecule converge       # apply the role to localhost
molecule idempotence    # run again: "Idempotence completed successfully."
molecule verify         # "Verifier completed successfully."
molecule test           # the whole sequence, from a clean start
```

- With `managed: false` there's nothing to create or destroy — Molecule uses this machine as `instance`.
- Because it's this machine, **clean up before a fresh test**: `rm -rf /tmp/inf345-p09` — or give the scenario a `prepare.yml` / `cleanup.yml` that does it.
- Before the fix, `molecule idempotence` fails with `Idempotence test failed because of the following tasks:` and lists the three tasks.

Block 5

Common mistakes

When it doesn’t work — idempotence

Symptom	Likely cause
`changed` on every run	`shell` / `command` with no `creates`, `removes` or `changed_when`
A line appears twice in a file	`echo >>` instead of `lineinfile` (or `lineinfile` without `regexp:`)
`changed` every run on a `file: state=touch`	Timestamps update — add `modification_time: preserve` and `access_time: preserve`
A handler runs every time	A task that notifies it isn’t idempotent
`--check` breaks a later task	A skipped `command` left an empty `register` — `check_mode: false` on read-only commands

When it doesn’t work — Molecule

Symptom	Likely cause
`the role 'webapp' was not found`	`ANSIBLE_ROLES_PATH` doesn’t point at `roles/` (relative to the scenario directory)
`Idempotence test failed because of the following tasks`	Those tasks report `changed` on run 2 — fix them, not the test
`Failed to connect to the host via ssh`	Missing `ansible_connection: local` for the unmanaged `instance`
verify passes but the result is wrong	Your assertions check tasks or nothing at all — assert the **outcome**
A test passes once, then fails	State left from the last run — clean `/tmp/...` first, or use `cleanup.yml`

Self-check

- A task is `command: cat /etc/hostname`. What does it need to be idempotent? (`changed_when: false` — it only reads)
- Which flag shows what a run *would* change, without changing anything? (`--check`; add `--diff` to see the lines)
- In which `molecule test` step would `echo "x" >> file` be caught? (`idempotence` — the second converge reports `changed`)
- What reads a file's content inside `verify.yml`? (`ansible.builtin.slurp` — then `b64decode`)

Block 6

Today’s practice

Practice 09 — Make it idempotent & test it with Molecule

The `webapp` role, after a teammate's "quick fixes" with `shell` and `command`. You:

1. find the tasks that report `changed` on every run
2. replace them with real modules — or `creates:` / `changed_when:`
3. keep the result the same (files under `/tmp/inf345-p09/app`)
4. finish the Molecule scenario in `molecule/default/` — default driver, `managed: false`, localhost, connection local
5. write `verify.yml` assertions for the result

```
rm -rf /tmp/inf345-p09
ansible-playbook site.yml           # works
ansible-playbook site.yml           # changed=0
ansible-playbook site.yml --check   # changed=0
molecule test                       # all steps pass
```

Same flow as always: accept "Practice 09" on Maru, work in your repo, push to `main`. Graded automatically on every push — the fixes, idempotence, check mode, and your Molecule scenario.

By the end of this lesson, you should be able to

- ☐ Explain idempotence, and read `ok` / `changed` in the recap
- ☐ Spot a non-idempotent task — and fix it with a module, `creates`, or `changed_when`
- ☐ Preview a run with `--check --diff`, and keep commands working in check mode
- ☐ Explain why a handler runs every time — and fix the real cause
- ☐ Write a Molecule scenario: `molecule.yml`, `converge.yml`, `verify.yml`
- ☐ Run `molecule converge`, `idempotence`, `verify` and `test`

Before next lecture

- ☐ Finish today’s practice if you didn’t wrap it up in session — same Maru flow as always
- ☐ **Lab 02 (RHA RH294)** is due **Wed, Nov 4, 23:59** — one week left; see "Red Hat labs" in Maru
- ☐ Run `ansible-lint` on your own role from last week — fix what it finds

Next week is Lab 02's deadline week — bring your questions.

Next lecture

Ansible in production patterns — inventories for real environments, secrets with Ansible Vault, and running automation safely at scale.